FARM MANAGEMENT SYSTEM (FMS)

USER STORY:

In the Farm Management System (FMS), as a farmer, farm manager, or system administrator, I want to use a centralized Salesforce-based platform to record and manage farm, crop, farmer, and buyer information so that all agricultural data is stored securely and systematically. I want to enter and update farm details, crop production records, and buyer transactions easily through a user friendly interface with proper validation rules to prevent incorrect entries. I also want the system to generate real-time reports and interactive dashboards so that I can analyze farm productivity, monitor sales, and make informed decisions. Additionally, I want role-based access and secure data storage to ensure that sensitive information is protected while enabling smooth collaboration among users. This system will help reduce manual paperwork, minimize errors, improve efficiency, and support data-driven farm management.

1. INTRODUCTION

1.1 Project Overview :

The Farm Management System (FMS) is a Salesforce-based application designed to digitize and streamline agricultural operations. It manages data related to farms, crops, farmers, and buyers while providing real-time insights through reports and dashboards. The system helps farmers and managers monitor agricultural activities efficiently and make informed decisions.

1.2 Purpose :

The purpose of FMS is to replace traditional manual farming records with an automated cloud-based solution. It eliminates data loss, provides inventory alerts, auto-calculates profit/loss, and offers mobile accessibility. It ensures data security via Salesforce and enables data-driven farming.

1.3 Existing System & Proposed System :

Existing System: Manual notebooks, Excel sheets - No security, Data loss, No reports.
Proposed System: Salesforce cloud platform with custom objects, automation, validation, reports and dashboards. Low-code, secure, scalable.

2. IDEATION PHASE

2.1 Problem Statement:

Farmers face issues like losing crop records, not knowing exact profit, manual calculation errors, no centralized data, and no low-stock alerts for seeds/fertilizers.

2.2 Empathy Map Canvas:

SAYS: I lose my crop records, I don't know my profit.
THINKS: I need a simple app, cheap and easy to use.
DOES: Writes in notebook, calls suppliers, manually calculates.
FEELS: Frustrated due to data loss, worried about expenses.
PAIN: Data loss, calculation mistakes, no central data, no alerts.
GAIN: Centralized data, auto calculation, dashboard, email alerts.

2.3 Brainstorming:

List of Ideas:
1. Manual farm records keeping
2. Excel-based management
3. Web-based farm portal
4. Mobile-based farm app
5. Salesforce-based Farm Management Syste

Idea Prioritization: We selected Salesforce-based FMS because it is Low-code, High Security, Cloud-based, Mobile Support, Real-time Reports, Free Developer Edition.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map:

Awareness: Farmer realizes need for digital system.
Exploration: Searches for solutions, sees Salesforce demo.
Onboarding: Creates Farmer profile, adds Farm details.
Usage: Adds Crops, Inventory items, Livestock.
Monitoring: Views Dashboard, checks crop status, low stock.
Sales: Harvests crop, system auto creates sales record, views profit.
Support: Gets email alerts, support via dashboard.

3.2 Solution Requirements:

Functional Requirements:
- Create Farmer, Farm, Crop, Inventory, Buyer, Sales objects.
- Establish Master-Detail and Lookup relationships.
- Validation Rules: Harvest_Date > Sowing_Date, Quantity > 0.
- Record-Triggered Flow for sales automation.
- Reports: Crop Yield, Low Inventory, Monthly Sales.
- Dashboard: Total Farmers, Crops by Status, Sales Trend.

Non-Functional Requirements:
- Should work on Desktop and Mobile (Salesforce Mobile App).
- Secure with Role-based access.
- Scalable to handle 1000+ farmers.
- User-friendly Lightning UI.
- Performance: Reports load within 3 seconds.

3.3 Data Flow Diagram:

Level 0 DFD: Farmer/Buyer (External Entity) -> FMS Process (Central System) -> Salesforce DB (Data Store) -> Reports/Dashboard (Output).
Level 1 DFD:
1.0 Farmer Management -> Farmer Object -> DB
2.0 Farm Management -> Farm Object -> DB
3.0 Crop Management -> Crop Object -> Flow -> Sales Object
4.0 Inventory Management -> Inventory Object -> Low Stock Check
5.0 Buyer & Sales Management -> Buyer/Sales Object -> Reports

3.4 Technology Stack:

Frontend: Salesforce Lightning Experience, Lightning App Builder.
Backend: Salesforce Platform, Custom Objects, Validation Rules, Flows.
Database: Salesforce Cloud Database.
Tools: Developer Org, Setup Menu, App Manager, Object Manager, Flow Builder, Report Builder.

4. PROJECT DESIGN

4.1 Problem-Solution Fit:

Problem: Data loss, no profit tracking, manual errors.
Customer Segment: Small and medium farmers, farm managers.
Existing Alternatives: Manual books, Excel - Not secure, not automated.
Solution: Salesforce FMS solves all pain points with automation, security, and low cost.

4.2 Proposed Solution:

We propose a Lightning Application named "Farm Management System (NM)" containing 5-6 custom objects.
Object Structure:
Farmer*c (Parent) - One to Many - Farm*c
Farm*c (Parent) - One to Many - Crop*c, Inventory*c, Livestock*c
Crop*c (Parent) - One to Many - Sales*c
Fields: Picklists for Crop Name (Rice, Wheat, Maize, Sugarcane, Cotton), Status (Sown, Growing, Harvested).
Formula Field: Total_Amount*c = Quantity_Sold*c * Rate_Per_Kg__c.
Automation: When Crop Status = Harvested, Flow creates Sales record and sends email.

4.3 Solution Architecture:

Architecture Diagram (3-Tier):
Presentation Tier: User Interface (Lightning Tabs, App, Mobile App)
Business Logic Tier: Validation Rules, Record-Triggered Flows, Approval Process
Data Tier: Custom Objects (Farmer, Farm, Crop, Inventory, Sales)

Architecture Flow: End User (Browser/Mobile) -> Salesforce Cloud Platform -> Lightning App -> Custom Objects -> Salesforce Database -> Reports & Dashboards -> End User.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning:

Sprint 1 (Week 1): Requirement Gathering, Empathy Map, Brainstorming, User Story Documentation.
Sprint 2 (Week 2): Create Salesforce Developer Org, Data Modeling, Create Custom Objects and Fields, Relationships.
Sprint 3 (Week 3): Create Custom Tabs, Create Lightning App, Create Validation Rules, Page Layouts.
Sprint 4 (Week 4): Build Flows, Create Reports and Dashboards, Testing, Documentation and GitHub Upload.

6. PROJECT DEVELOPMENT PHASE

6.1 Development Steps:

Step 1: Created Salesforce Developer Org from http://developer.salesforce.com.
Step 2: Created Custom Objects:
Farmer*c: Name (Text), Phone*c (Phone), Email*c (Email), Land_Size*c (Number), Address__c (Long Text Area)
Farm*c: Farm_Name*c (Text), Location*c (Text), Area*c (Number), Farmer*c (Master-Detail to Farmer*c)
Crop*c: Crop_Name*c (Picklist), Sowing_Date*c (Date), Harvest_Date*c (Date), Status*c (Picklist), Expected_Yield*c (Number), Farm*c (Lookup to Farm*c)
Inventory*c: Item_Name*c (Text), Category*c (Picklist: Seeds, Fertilizer, Pesticide, Equipment), Quantity*c (Number), Unit_Price*c (Currency), Supplier*c (Text), Farm__c (Lookup)
Sales*c: Crop*c (Lookup), Quantity_Sold*c (Number), Rate_Per_Kg*c (Currency), Total_Amount*c (Formula), Buyer_Name*c (Text), Sale_Date__c (Date)

Step 3: Created Custom Tabs for all objects via Setup -> Tabs.

Step 4: Created Lightning App: Setup -> App Manager -> New Lightning App -> Name: Farm Management System (NM) -> Add all Tabs -> Assign to System Administrator Profile.
Step 5: Validation Rules: On Crop*c: Harvest_Date*c > Sowing_Date*c else error, Quantity*c > 0.
Step 6: Created Record-Triggered Flow: Trigger when Crop*c Status = Harvested, Action: Create Record in Sales*c + Send Email Alert to Farmer.
Step 7: Created Reports: Crop Yield Report, Low Inventory (<10 Qty), Monthly Sales Report.
Step 8: Created Dashboard: 4 Components - 1. Total Farmers (Metric), 2. Crops by Status (Donut Chart), 3. Low Stock Inventory (Bar Chart), 4. Monthly Sales (Line Chart).
Step 9: Added Sample Data - 20 Farmers, 30 Farms, 50 Crops.

7. FUNCTIONAL AND PERFORMANCE TESTING

7.1 Functional Testing:

TC01: Create Farmer record with valid data - Expected: Saved - Result: Passed
TC02: Create Farm with Farmer link - Expected: Master-Detail works - Result: Passed
TC03: Enter Harvest Date less than Sowing Date - Expected: Validation error - Result: Passed (Error shown)
TC04: Quantity negative - Expected: Validation error - Result: Passed
TC05: Change Crop Status to Harvested - Expected: Flow auto creates Sales - Result: Passed
TC06: View Dashboard - Expected: Charts display - Result: Passed
TC07: Mobile App access - Expected: App opens in Salesforce mobile - Result: Passed

7.2 Performance Testing:

Tested with 100 Farmer records, 200 Crop records - Page load < 2 sec, Report load < 3 sec, Dashboard refresh < 4 sec - All Passed.

8. ADVANTAGES & DISADVANTAGES

Advantages:

1. Centralized cloud-based data management.
2. Low-code platform - No need for heavy coding.
3. Real-time reports and dashboards for decision making.
4. Mobile accessibility via Salesforce Mobile App.
5. Automation reduces manual work and errors.
6. Highly secure with Salesforce Shield.
7. Scalable for thousands of farmers.
8. Free Developer Edition for learning and prototype.

Disadvantages:

1. Requires internet connectivity - No offline mode.
2. Salesforce Enterprise licenses are costly.
3. Storage limits in developer edition (5MB data, 20MB file).
4. Farmers need basic digital literacy.
5. Limited customization compared to full-code apps.

9. CONCLUSION

The Farm Management System (FMS) project has been successfully designed and implemented on the Salesforce platform as part of Naan Mudhalvan initiative. It fulfills all core objectives of digitizing farm operations, providing secure storage, and generating real-time insights. The project gave us deep practical knowledge in Salesforce Administration, Data Modeling, Automation (Flows), Reports & Dashboards. This system is real-world applicable and can be extended to help the agricultural sector in Tamil Nadu.